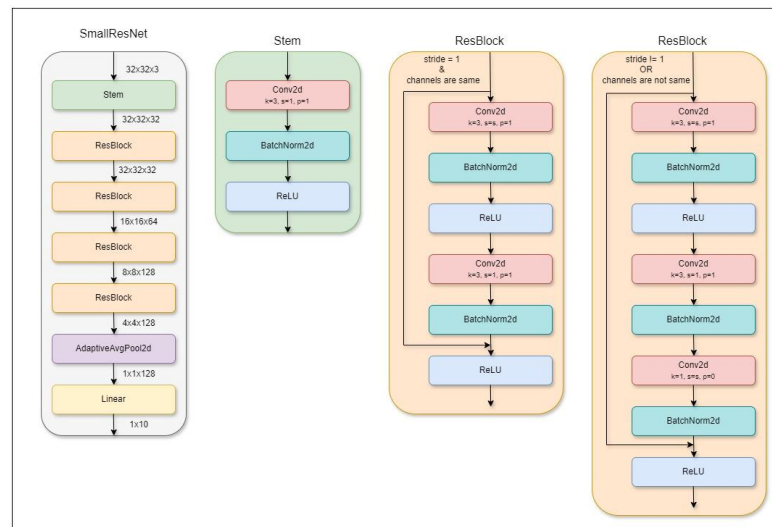


1. Parameters

Stem	Conv: $3 \times 32 \times 3 \times 3 = 864$
	BN: $32 \times 2 = 64$
ResBlock1	Conv1: $32 \times 32 \times 3 \times 3 = 9,216$
	BN1: $32 \times 2 = 64$
	Conv2: $32 \times 32 \times 3 \times 3 = 9,216$
	BN2: $32 \times 2 = 64$
ResBlock2	Conv1: $32 \times 64 \times 3 \times 3 = 18,432$
	BN1: $64 \times 2 = 128$
	Conv2: $64 \times 64 \times 3 \times 3 = 36,864$
	BN2: $64 \times 2 = 128$
	Skip Conv: $32 \times 64 \times 1 \times 1 = 2,048$
	Skip BN: $64 \times 2 = 128$
ResBlock3	Conv1: $64 \times 128 \times 3 \times 3 = 73,728$
	BN1: $128 \times 2 = 256$
	Conv2: $128 \times 128 \times 3 \times 3 = 147,456$
	BN2: $128 \times 2 = 256$
	Skip Conv: $64 \times 128 \times 1 \times 1 = 8,192$
ResBlock4	Conv1: $128 \times 128 \times 3 \times 3 = 147,456$
	BN1: $128 \times 2 = 256$
	Conv2: $128 \times 128 \times 3 \times 3 = 147,456$
	BN2: $128 \times 2 = 256$
	Skip Conv: $128 \times 128 \times 1 \times 1 = 16,384$
FC	$128 \times 10 + 10$ (bias) = 1,290
Total	620,714



```
[device] CPU, threads=8
[mean] [0.49165572255588375, 0.4823340590957845, 0.4466792409572851]
[std ] [0.2470811602785194, 0.24350869632987013, 0.26158066485723697]
[params] 620714
```

Number of parameters calculated by hand were computed as 620714. Number of parameters framework-reported was 620714. Since it is same, we can expect that manually calculated parameters are correctly calculated. Also, AdaptiveAvgPool does not have learnable parameters since it is just calculating the average value within specific window.

2. BatchNorm

2.1. Explain the parameters it learns (per-channel scale γ and shift β)

Basically, when performing BatchNorm, the data distribution is computed to have a mean of 0 and a variance of 1. In this case, the original distribution's mean value is ignored, eliminating the bias-like effect, and the variance is also limited to 1, restricting the data's expressiveness. Therefore, by using β and γ , the mean and variance are scaled and shifted, improving the central information and the data's expressiveness. $\text{Batch Norm} \rightarrow \hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}, y = \gamma \hat{x} + \beta$

γ is applied as a scale factor that adjusts the variance of the normalized data per channel, changing the variance from 1 to γ^2 when applied.

β is an offset that adjusts the mean of the normalized data per channel, changing the mean from 1 to β when applied.

2.2. Describe the optimization effects

The purpose of BatchNorm is to mitigate Internal Covariate Shift, thereby stabilizing the gradient. So, it prevents vanishing/exploding gradient which is caused by saturation in activation functions. In addition, applying BatchNorm provides the following benefits.

- 2.2.1. Makes training deep networks much easier
- 2.2.2. Allows higher learning rates and faster convergence
- 2.2.3. More robust to initialization
- 2.2.4. Performs regularization during training
- 2.2.5. Can be fused with Conv, enabling faster operation during Inference. This is because BN uses fixed statistics during inference.

2.3. Clarify how behavior differs in train vs eval mode

During Train, the mean and variance are calculated in real-time per batch for normalization, and simultaneously, the running mean and running var of the entire dataset are updated. However, if the batch size is small, the statistics can be unstable. During Eval, instead of mini-batch statistics, the accumulated running mean and running var from training are used. Therefore, since there is no real-time calculation, fusion with Conv is possible. However, if the distribution of training data and test data differs, performance degradation occurs.

3. Scheduler

3.1. Explain how the cosine_warmup_scheduler works, composed of two parts—linear warm-up and cosine decay

The cosine_warmup_scheduler is composed of two parts to mitigate instability in the early stages of training and promote convergence in the later stages. In the early part, Linear Warm-up is applied, which is a stage that linearly increases the lr factor from 0 to 1 in the early training phase. This prevents instability (gradient explosion) caused by an initially too high learning rate. In the later part, Cosine decay is applied, which is a stage that smoothly decreases the learning rate using a cosine function after warm-up. This helps prevent overfitting in the later stages and aids convergence to the optimal point.

3.2. Describe each part's mechanism

3.2.1. linear warm-up

In the early stages of learning, the lr starts low to allow the model to gradually adapt to the data. With each step, the global step increases, and when the warmup steps are reached, the lr factor becomes 1. That is, $lr\ factor = \frac{global\ step}{warmup\ step}$. The global step increases every time the scheduler function is called. Additionally, warmup steps is the total number of steps in the warm-up phase, and total steps is the total number of steps in the training. Therefore, warmup steps and total steps are fixed during training.

3.2.2. cosine decay

It gradually decreases the lr using the smooth curve of the cosine function. There is an advantage that learning becomes more stable because abrupt lr reduction does not occur. At the moment the warm-up ends, it starts from the lr factor achieved in the warm-up, and as the global step increases, it decreases the lr factor by inputting the progress rate into the cosine function. To prevent the learning rate from dropping to zero, it stops decaying at a predefined minimum learning rate factor.

$$\alpha_t = \alpha_0 \left(\eta + (1 - \eta) \cdot \frac{1}{2} \left(1 + \cos \left(\frac{t\pi}{T} \right) \right) \right) = \alpha_0 \left(\eta + (1 - \eta) \cdot \frac{1}{2} (1 + \cos(\pi \times progress)) \right), progress = \frac{global\ step - warmup\ step}{total\ step - warmup\ step}$$

3.3. Expected effects of using the scheduler

With Warm-up, initial instability due to random initialization of weights is prevented. Thus, it prevents exploding/vanishing gradient updates. With cosine decay, stable convergence in the later stages is made possible. So, it prevents diverge/oscillating gradient updates. Additionally, since the lr starts small due to the lr factor and gradually changes to the set lr, it is basically possible to utilize a high lr. Therefore, training can proceed quickly. Due to Cosine decay, the lr in the later stages is gradually reduced, helping to converge stably without divergence or oscillation. Also, subtle lr fluctuations add small variability to model updates, preventing overfitting.

4. Optimizer(Adam)

4.1. Explain Adam's mechanism with first/second-moment estimates (m , v)

4.1.1. Momentum

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

$$x_{t+1} = x_t - \alpha v_{t+1}$$

Momentum accumulates v_t (velocity) using the running mean of historical gradients. Therefore, depending on the size of ρ , if the gradient moves in the same direction, it accelerates the gradient change, and if it moves in a different direction, it suppresses the gradient change. Thus, this prevents oscillation or divergence.

4.1.2. RMSProp

$$S_t = \beta S_{t-1} + (1 - \beta) g_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{S_t} + \epsilon} g_t$$

S_t is the moving average of gradient squared, updating the learning rate considering the influence of historical gradients. Through this, adaptively, if the gradient in a specific direction is large, it reduces the learning rate in that direction to prevent divergence or oscillation, and if the gradient is small, it increases the learning rate to accelerate learning.

4.1.3. Adam

$$g_t = \nabla_{\theta} J(\theta_{t-1}) \quad (\text{Compute the gradient of } J \text{ w.r.t. } \theta \text{ at timestep } t)$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{Update biased first moment estimate})$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{Update biased second moment estimate})$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (\text{Compute bias - corrected first moment estimate})$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (\text{Compute bias - corrected second moment estimate})$$

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (\text{Update parameters})$$

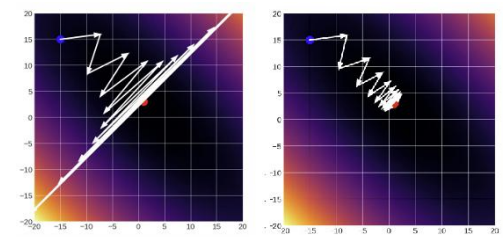
Adam combines the advantages of Momentum and RMSProp, using the first-moment estimate (m_t) and second-moment estimate (v_t) of the gradient. For the first moment, based on Momentum, it uses the moving average of historical gradients to maintain the directionality of past gradients. For the second moment, based on RMSProp, it uses the moving average of the squares of historical gradients to adaptively change the learning rate according to past gradients.

Additionally, initially, because β_1 and β_2 are large, m_t and v_t have values close to 0. Therefore, in the beginning learning does not proceed well, so bias correction is used to make the values of m_t and v_t larger initially.

4.2. How this differs from plain SGD's single global learning rate

$$x_{t+1} = x_t - \alpha \nabla f(x_t) \rightarrow \text{plain SGD}$$

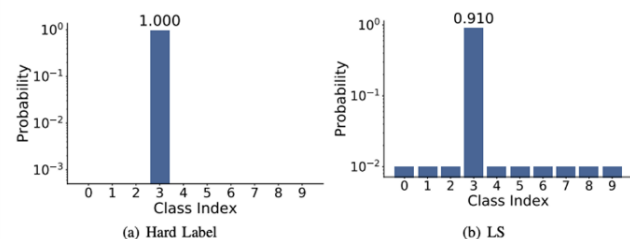
Plain SGD updates the parameter values using only the mini-batch gradient. In the case of Adam, it updates parameters adaptively, but in the case of SGD, as can be seen from the above equation, it updates parameters using only a single learning rate (α) and the current gradient ($\nabla f(x_t)$). Since SGD depends only on a single learning rate and the current gradient, it can diverge or oscillate as shown in the right figure. However, in the case of Adam, it reflects the moving average of historical gradients to suppress abrupt changes in the direction of the gradient and changes the learning rate. This enables more stable convergence.



5. Label Smoothing

5.1. Explain Label Smoothing's mechanism with respect to probability distribution

Label Smoothing converts the original hard label (one-hot encoded) to a soft label and calculates the loss between the model's predicted probability distribution and the target distribution. That is, Label Smoothing softens this one-hot encoded label by ϵ , lowering the probability of the correct class to $1 - \epsilon$, and evenly distributing $\epsilon / (\text{number of classes} - 1)$ to the remaining classes.



5.2. Describe the expected effect of using label smoothing

First, it prevents the model from making predictions with 100% confidence and injects a bit of uncertainty into learning. Through this, overconfidence can be avoided. Second, by smoothing the target distribution, it leads to flexible predictions rather than extreme ones, improving calibration and generalization performance. Third, if there is noise in the training data, hard labels cause the model to learn to assign high probabilities to this noise as well. Label smoothing can suppress learning noise. Finally, with hard labels, even small prediction errors cause a sharp increase in loss, making the loss landscape steep and rugged. Using soft labels makes the loss landscape flat and smooth, allowing for stable learning.

6. Discussion about using Step Decay vs. Cosine Decay

Using Step Decay instead of Cosine Decay shows almost same accuracy. So, we can expect it is just a design choice what kind of decay to use for training.

7. References: lecture5 Neural Networks 2.pdf (by prof.김중유)

Step Decay Result	Cosine Decay Result
<pre> mean 0.4934057225508175, 0.482134676957645, 0.4466752489572851 std 0.2430811660785194, 0.2435806963287013, 0.26158066485723697 [mean] 0.00714 [0/10] train loss 1.7895 acc 0.412 val loss 1.7514 acc 0.460 196.1s [ckpt] saved (val acc=0.460) [0/20] train loss 1.7368 acc 0.523 val loss 1.3344 acc 0.639 197.2s [ckpt] saved (val acc=0.639) [0/30] train loss 1.2308 acc 0.707 val loss 1.2120 acc 0.690 222.8s [ckpt] saved (val acc=0.690) [0/40] train loss 1.0628 acc 0.762 val loss 1.0374 acc 0.767 232.0s [ckpt] saved (val acc=0.767) [0/50] train loss 0.9883 acc 0.799 val loss 1.0249 acc 0.787 263.7s [ckpt] saved (val acc=0.799) [0/60] train loss 0.9699 acc 0.814 val loss 0.9655 acc 0.812 222.8s [ckpt] saved (val acc=0.812) [0/70] train loss 0.9405 acc 0.834 val loss 0.9223 acc 0.828 176.2s [ckpt] saved (val acc=0.834) [0/80] train loss 0.8708 acc 0.889 val loss 0.9105 acc 0.831 168.1s [ckpt] saved (val acc=0.889) [0/90] train loss 0.8581 acc 0.858 val loss 0.8939 acc 0.842 164.3s [ckpt] saved (val acc=0.889) [10/10] train loss 0.8376 acc 0.888 val loss 0.8780 acc 0.857 162.2s [ckpt] saved (val acc=0.897) [total] loss 0.8773 acc 0.869 </pre>	<pre> mean 0.4934057225508175, 0.482134676957645, 0.4466752489572851 std 0.2430811660785194, 0.2435806963287013, 0.26158066485723697 [mean] 0.00714 [0/10] train loss 1.7895 acc 0.412 val loss 1.7514 acc 0.460 192.7s [ckpt] saved (val acc=0.460) [0/20] train loss 1.3768 acc 0.524 val loss 1.3154 acc 0.540 115.8s [ckpt] saved (val acc=0.540) [0/30] train loss 1.1609 acc 0.709 val loss 1.1722 acc 0.706 116.3s [ckpt] saved (val acc=0.706) [0/40] train loss 1.0632 acc 0.762 val loss 1.0014 acc 0.745 116.8s [ckpt] saved (val acc=0.762) [0/50] train loss 1.0005 acc 0.790 val loss 1.0204 acc 0.785 121.0s [ckpt] saved (val acc=0.790) [0/60] train loss 0.9693 acc 0.816 val loss 0.9531 acc 0.817 119.8s [ckpt] saved (val acc=0.817) [0/70] train loss 0.9009 acc 0.833 val loss 0.9414 acc 0.821 115.1s [ckpt] saved (val acc=0.833) [0/80] train loss 0.8458 acc 0.884 val loss 0.8717 acc 0.852 118.4s [ckpt] saved (val acc=0.884) [10/10] train loss 0.8282 acc 0.871 val loss 0.8678 acc 0.855 110.3s [ckpt] saved (val acc=0.895) [total] loss 0.8773 acc 0.869 </pre>